Reviewer Pack — Noncommutative Rank Gap in Read-Twice Branching Programs

Entry f245272528b8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 00:52:33 UTC

Noncommutative Rank Gap in Read-Twice Branching Programs

Entry ID: `f245272528b8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 00:52:33 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For a read-twice BP P over n variables, the noncommutative rank of its transition matrix M satisfies rank_nc(M) ≥ Ω(n), whereas for read-once BPs, rank_nc(M) = O(log n).

Rationale (proposer's reasoning):

> Noncommutative rank captures algebraic dependencies in matrix factorizations, which may reflect the repeated variable access in read-twice BPs. This could expose structural asymmetries in their computational power compared to read-once BPs.

Taxonomy category: `BP_READTWICE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `85b34d78704b8192`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A, p):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        factor = -A[i][i] % p
        for j in range(n):
            A[i][j] = (A[i][j] * factor) % p
        for j in range(n):
            if i != j:
                factor = A[j][i] % p
                for k in range(n):
                    A[j][k] = (A[j][k] - factor * A[i][k]) % p

def rank_nc(A, p):
    n = len(A)
    rank = 0
    gaussian_elimination(A, p)
    for i in range(n):
        if A[i][i] != 0:
            rank += 1
    return rank

def generate_bp(n, read_twice=False):
    bp = []
    for _ in range(n):
        if read_twice:
            access = random.sample(range(2), 2)
        else:
            access = [random.randint(0, 1)]
        bp.append(access)
    return bp

def tensor_product(bp1, bp2):
    n1 = len(bp1)
    n2 = len(bp2)
    result = [[[] for _ in range(n2)] for _ in range(n1)]
    for i in range(n1):
```

```
        for j in range(n2):
            for a in bp1[i]:
                for b in bp2[j]:
                    result[i][j].append((a, b))
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20
    p = 23  # Prime number for modulo operation
    instances_tested = 0
    total_rank = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(10):  # Test with 10 random read-once and read-twice BPs
        bp1 = generate_bp(n, False)
        bp2 = generate_bp(n, True)
        M = tensor_product(bp1, bp2)
        rank = rank_nc(M, p)
        instances_tested += 1
        total_rank += rank

    avg_rank = total_rank / instances_tested
    if read_twice:
        if avg_rank <= math.log(n, 2):
            conjecture_holds = False
            counterexample = "Read-twice BP has noncommutative rank ≤ log n"
    else:
        if avg_rank > math.log(n, 2):
            conjecture_holds = False
            counterexample = "Read-once BP has noncommutative rank > log n"

    return {
        "metric_name": "Noncommutative Rank",
        "metric_value": avg_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=NA support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
```

```
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ae8bb1cc.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ae8bb1cc.py", line 78, in run_trial
    rank = rank_nc(M, p)
           ^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ae8bb1cc.py", line 38, in rank_nc
    gaussian_elimination(A, p)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ae8bb1cc.py", line 23, in gaussian_elim
    if abs(A[j][i]) > abs(A[max_row][i]):
       ^^^^^^^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError in Gaussian elimination, preventing result collection. Support condition cannot be evaluated without successful trials. | next: Debug Gaussian elimination implementation to handle matrix entries correctly (ensure numerical values instead of lists)

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112955
2	preregistration	ollama_remote	qwen3:8b	0	0	32001
3	novelty	ollama_remote	qwen3:8b	0	0	27695
4	novelty	ollama_remote	qwen3:8b	0	0	16862
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25643
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15837
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14953
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11083
9	judge	ollama_remote	qwen3:8b	0	0	26902

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 283931 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f245272528b8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f245272528b8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f245272528b8.tar.gz` (if generated)